Python Assignment Questions

Part 1 – Output & Data Types

Q1. Print name, age, favorite color + show type of age

- **Concepts:** `print()`, variables, `type()`, newline `\n` (or separate print statements)
- **Hint:** Store each value in a variable, then use `print(var1, var2, var3, sep='\n')` or three `print()` calls. For `type(age)`, wrap it inside `print()`.
- **Real-world (Indian companies):**
  - **Infosys / TCS** – Debugging logs: `print()` is used in internal scripts to track variable types during data cleaning.
  - **Zomato** – Checking types of price values received from restaurant APIs (int vs string).

Q2. Predict and verify `"10"+"10"`, `10+10`, `10+int("10")`

- **Concepts:** String concatenation (`+`), integer addition, type conversion (`int()`)
- **Hint:** Remember: `"10"` is text, `10` is number. `int("10")` converts text to number.
- **Real-world:**
  - **Flipkart** – When extracting product quantities from user input (string like `"2"`), they convert to int before adding to cart.
  - **Paytm** – Adding two invoice amounts (strings from different sources) requires conversion to float/int.

Part 2 – User Input & f-strings

Q3. Ask name, birth year, city → calculate age → print f-string

- **Concepts:** `input()`, `int()` conversion, arithmetic (current year – birth year), f-strings
- **Hint:** Use `2026` as current year or import `datetime` for dynamic year.
- **Real-world:**

- **Freshworks (Chennai)** – Customer onboarding forms: collect birth year to send birthday discounts.
- **Byju's** – Student registration: calculate age to recommend appropriate courses.

Q4. Repeat password prompt until `"python123"` → `"Access granted"`

- **Concepts:** `while` loop, string comparison, `input()`
- **Hint:** Initialize `pin = ""`. Use `while pin != "python123"` and inside ask for input.
- **Real-world:**
  - **HDFC Bank** – ATM PIN validation (limited attempts, but same logic).
  - **Razorpay** – API key verification before allowing access to payment dashboard.

Part 3 – Lists & Loops

Q5. List of 5 fruits → print third fruit → slice second to fourth

- **Concepts:** List creation, indexing (0-based), slicing `[start:end]` (end exclusive)
- **Hint:** Third fruit = index 2. Second to fourth = indices 1 to 3 (`[1:4]`).
- **Real-world:**
  - **BigBasket** – Displaying top 3 best-selling items (slicing product list).
  - **Swiggy** – Restaurant menu: show "starters" section using list slicing.

Q6. Loop through `temperatures` and print each with `"°C"`

- **Concepts:** `for` loop, iterating over list, `print()` with concatenation or f-string
- **Hint:** `for temp in temperatures: print(f"{temp}°C")`
- **Real-world:**
  - **Weather monitoring at Indian Railways** – Loop over sensor temperature readings to check for overheating.
  - **Tata Power** – Daily temperature logs from transformers.

Q7. Print even numbers from 1 to 20 using `for` and `range()`

- **Concepts:** `range(start, stop, step)`, `for` loop, modulus `%` or step argument
- **Hint:** Use `range(2, 21, 2)` or `if i % 2 == 0`.
- **Real-world:**
  - **PhonePe** – Generating even-numbered transaction IDs for a particular batch.
  - **Ola Cabs** – Assigning drivers to even-numbered slots in a shift roster.

Part 4 – Conditionals

Q8. Positive / Negative / Zero using `if-elif-else`

- **Concepts:** Comparison operators (`>`, `<`, `==`), `if-elif-else`
- **Hint:** Check `num > 0`, then `num < 0`, then `else`.
- **Real-world:**
  - **Cred** – Checking credit score change: positive change → reward, negative → alert.
  - **Groww** – Stock price movement: display “up” / “down” / “unchanged”.

Q9. Grade based on marks (>=90 A, >=75 B, >=50 C, else D)

- **Concepts:** `elif` ladder, relational operators, order of conditions
- **Hint:** Start from highest mark condition downwards.
- **Real-world:**
  - **upGrad** – Student assignment grading system.
  - **AMCAT (Aspiring Minds)** – Automating test result grading for companies.

Part 5 – Functions

Q10. Define `greet(name)` and call with three different names

- **Concepts:** `def`, parameter, argument, calling a function
- **Hint:** `def greet(name): print(...)` then `greet("Ravi")`, etc.
- **Real-world:**
    - **Amazon India** – Sending personalised email greetings (function called for each customer name).
    - **MakeMyTrip** – "Welcome back {name}" on app login.

Q11. Function `calculate_area(length, width)` that returns area

- **Concepts:** `return` statement, function with parameters, calling and printing returned value
- **Hint:** Inside function: `area = length * width` then `return area`.
- **Real-world:**
    - **L&T Construction** – Calculating material required for rectangular slabs.
    - **Urban Company** – Computing price for carpet cleaning based on room dimensions.

Part 6 – Dictionaries

Q12. Dictionary for book → print author

- **Concepts:** Dictionary literal `{}`, key-value pairs, accessing value by key
- **Hint:** `book = {"title":"...", "author":"...", ...}` then `print(book["author"])`.
- **Real-world:**
    - **Goodreads (owned by Amazon India team)** – Storing book metadata.
    - **Pratilipi** – User-uploaded stories dictionary with author, language, genre.

Q13. Ask user for product name & price → store in dict → display

- **Concepts:** `input()`, dynamic dictionary assignment, f-string
- **Hint:** Create empty dict, then `product["name"] = input(...)`, `product["price"] = input(...)`.
- **Real-world:**
  - **Meesho** – Adding a new product to seller dashboard (temporary dict before API call).
  - **Blinkit** – Grocery item creation form.

Part 7 – Debugging & Argument Order

Q14. Fix the `bio_data(25, "Ravi", "Mumbai")` error

- **Concepts:** Function parameter order, argument matching, missing default values
- **Hint:** The function expects `(name, age, city)` but caller passes `(25, "Ravi", "Mumbai")`. Swap arguments or reorder parameters. Also fix potential missing call if line is last.
- **Real-world:**
  - **Wipro** – Debugging internal APIs where arguments are passed in wrong order (e.g., swapping customer ID and order ID).
  - **Zeta (banking tech)** – Parameter misorder causes transaction failures in test environments.

Part 8 – Mini Project (Employee Directory)

Q15. Employee directory using dictionary + while loop + for loop

- **Concepts:** `while` loop with quit condition, dictionary assignment, `for` loop to iterate dict items

Hint:

```
employees = {}
while True:
    name = input("Enter name (or 'quit'): ")
    if name == 'quit':
```

```
        break
    dept = input("Enter department: ")
    employees[name] = dept

for name, dept in employees.items():
    print(f"Employee: {name} | Department: {dept}")
```

- **Real-world (Indian companies):**
    - **Infosys** – Internal HR tool to build a temporary team directory from user input.
    - **Gupshup (messaging platform)** – Mapping employee names to their reporting manager.
    - **Nykaa** – Store manager creating a shift roster (employee → department assignment).
